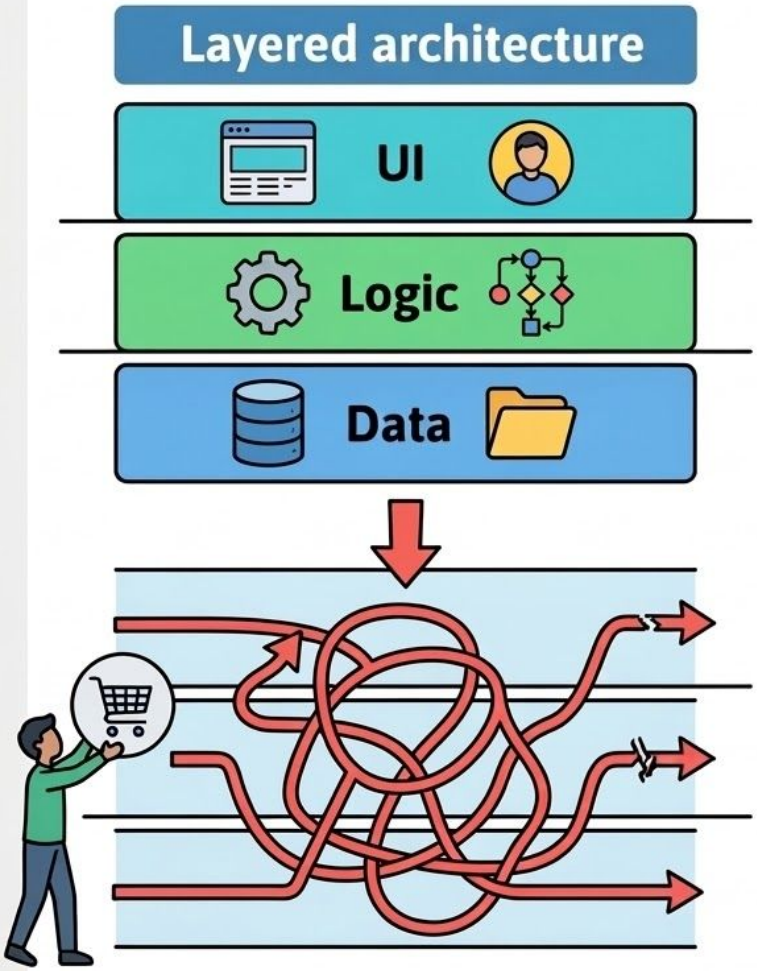
UNIVERSITY OF SZEGED
Department of Software Engineering
UNIVERSITAS SCIENTIARUM SZEGEDIENSIS

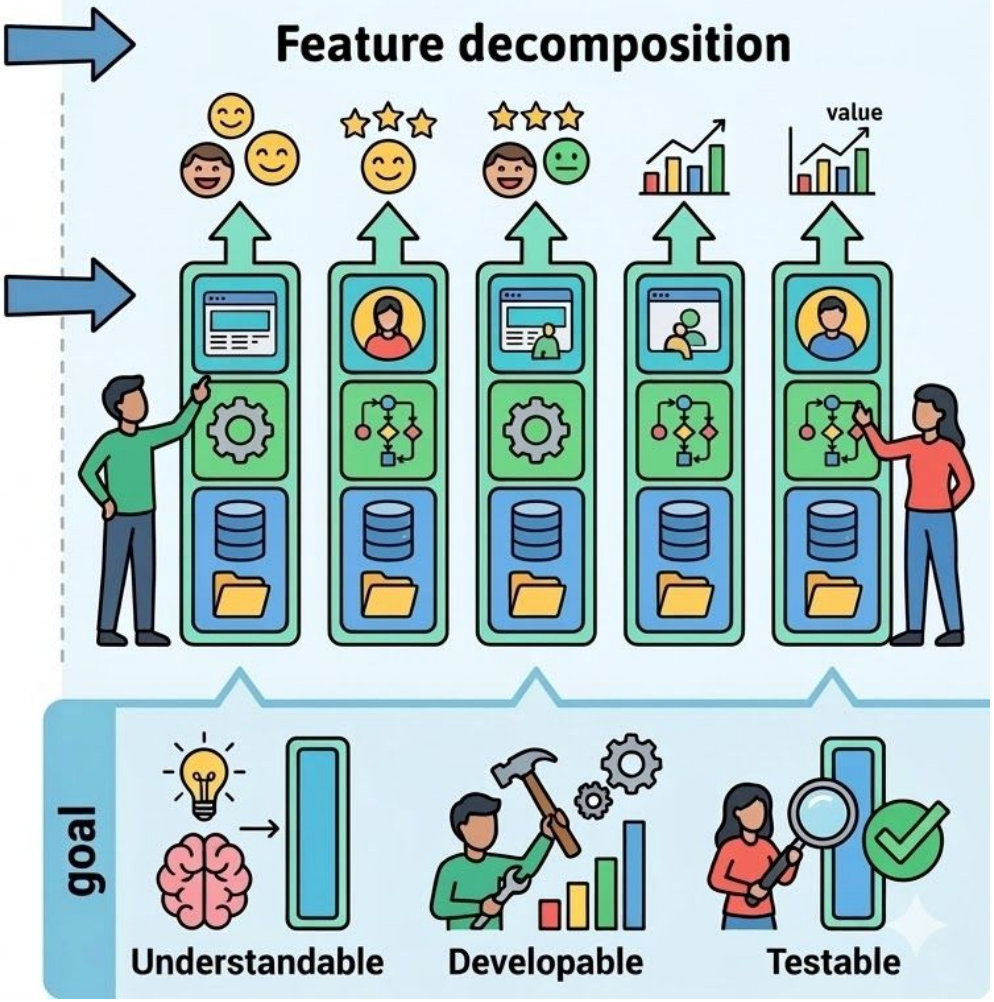
Software Architectures

Practice 4

Zoltán Richárd JÁNKI, PhD

Feature Decomposition





Features

- Feature = a unit of functionality that delivers observable value to a stakeholders
- Feature ≠ User Story
- Feature ≠ Module
- User Story < Feature < Module

- Good litmus test: "Can a user or stakeholder describe this in one sentence without technical jargon?"
- Examples: "Search for products", "Export report as PDF", "Two-factor login"

Features (example: EventHub)

- Event Organizers: to create and manage events (concerts, conferences, workshops)
- Customers: to browse events, select seats/tickets, and complete purchases
- Administrators: to oversee the platform, manage users, and view analytics

Feature vs. Component vs. Module vs. Service

Concept	Slicing Axis	Scope	Example
Feature	User value (vertical)	Cross-cutting	“Shopping cart”
Component	Technical responsibility	Single concern	CartWidget (UI)
Module	Code organization	Package/folder	cart/ directory
Service	Deployment boundary	Runtime process	cart-service

UNIVERSITAS SCIENTIARUM SZEGEDIENSIS
UNIVERSITY OF SZEGED
Department of Software Engineering

The Decomposition Process

1. Identify - gather raw features from requirements / stakeholders
2. Classify - core vs. supporting vs. generic
3. Decompose - break large features into sub-features
4. Define boundaries - decide what data and behaviour belongs where
5. Map to architecture - assign features to layers / components / services

Step 1: Identify Features

- Start from user-facing capabilities (not from the data model)
- Sources: user stories, event storming results, UI mockups, stakeholder interviews
- Tip: verb + noun pattern
  - manage inventory
  - generate invoice
  - authenticate user
- Output: a flat, unsorted feature list

Step 2: Classify Features

Borrowed from Domain-Driven Design strategic thinking

- Core features - what makes this product unique, highest business value (invest the most here)
- Supporting features - necessary but not differentiating (e.g.: user management)
- Generic features - commodity, often buy or reuse (e.g.: email sending, logging)

Why classify?
- build vs. buy decision
- where to spend architecture effort

Step 3: Decompose Large Features

- Apply the MECE principle
  - Mutually Exclusive, Collectively Exhaustive
  - Each sub-feature should have a single responsibility at the feature level
  - Do decomposing iteratively
    - Termination condition: if a sub-feature can be implemented in one iteration

Example:
Feature: managing orders
Sub-features: create order, track order status, cancel/modify order, order history & filtering

Step 4: Define Feature Boundaries

- Answer 4 questions for each (sub-)feature

- Data ownership: Which data does this feature own vs. reference?
- Input/Output: What triggers it? What does it produce?
- Dependencies: Which other feature does it need?
- Invariants: What business rules must always hold?

Boundaries/Features	Manage orders - Create order
Owns/References	Order/Product Catalog, Authentication
Input/Output	Place order button / Order record with pending status
Dependencies	Authentication, Product Catalog, Shopping Cart
Invariants	Order total > 0, All items in stock, Valid shipping address required, Prices are valid

Step 5: Map Features to Architecture

- Layered architecture: A feature touches UI, logic and data
- Component-based architecture: Each feature is a component

Pick the primary dimension!

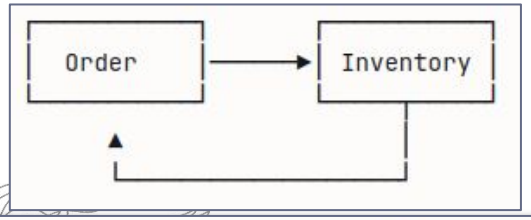
```
src/
├── features/
│   ├── order/
│   │   ├── ui/
│   │   │   ├── CreateOrderForm
│   │   │   └── OrderStatusView
│   │   ├── logic/
│   │   │   ├── createOrder
│   │   │   └── orderValidation
│   │   └── data/
│   │       └── orderRepository
│   ├── product/
│   │   ├── ui/
│   │   ├── logic/
│   │   └── data/
│   └── cart/
│       ├── ui/
│       ├── logic/
│       └── data/
│
└── shared/                          ← Cross-cutting concerns
    ├── auth/
    ├── logging/
    └── errorHandling/
```

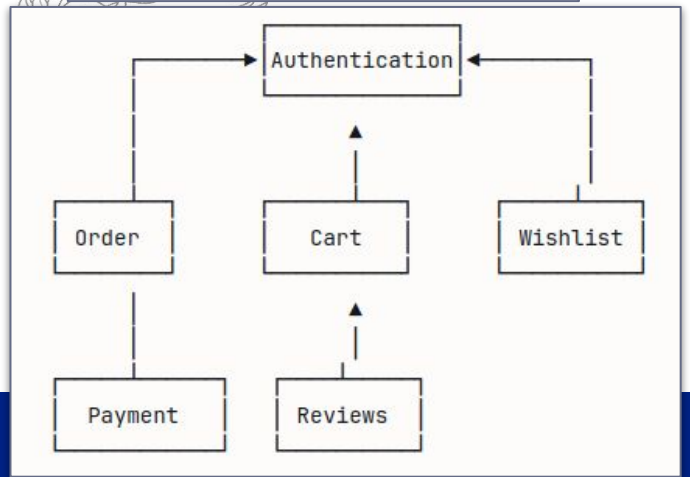
```
src/
├── ui/                              ← UI layer
│   ├── order/
│   │   ├── CreateOrderForm
│   │   ├── OrderStatusView
│   │   └── OrderHistoryList
│   ├── product/
│   │   ├── ProductList
│   │   └── ProductDetail
│   └── cart/
│       ├── CartView
│       └── CartSummary
│
├── logic/                           ← Business Logic layer
│   ├── order/
│   │   ├── createOrder
│   │   ├── cancelOrder
│   │   └── orderValidation
│   ├── product/
│   │   └── productSearch
│   └── cart/
│       ├── addToCart
│       └── calculateTotal
│
└── data/                            ← Data layer
    ├── order/
    │   └── orderRepository
    ├── product/
    │   └── productRepository
    └── cart/
        └── cartRepository
```

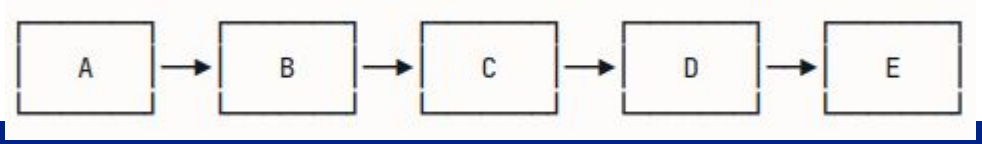
UNIVERSITY OF SZEGED
UNIVERSITAS SCIENTIARUM SZEGEDIENSIS
Department of Software Engineering

Feature Dependency Map

- After decomposition, draw a feature dependency graph
- Look for…
  - cycles (bad)
  - god-features (too many dependants)
  - orphans (maybe unnecessary)
- It may also decide the implementation order







Common Pitfalls

- Too fine-grained - decomposing into individual CRUD operations loses the “user value” perspective
- Data-driven decomposition - slicing by database tables instead of behaviour
- Ignoring cross-cutting concerns - auth, logging and error handling are not features; they are infrastructure
- Premature service boundaries - don’t turn every feature into a microservice on day one